

Adding Writable Network Number Support to BACnet Virtual Router

This guide explains how to add writable `Network_Number` property support to both the BACnet IP network port object and virtual network port objects in a CAS BACnet Stack virtual router implementation. It assumes you have already implemented the basic virtual router with virtual devices.

Overview

The `Network_Number` property (property identifier 425) on a Network Port object is optionally writable. When a BACnet client writes to it, the device must:

1. Store the pending new value.
2. Set `Changes_Pending` to `true`.
3. Wait for a `ReinitializeDevice` request to apply and broadcast the change. Three reinitialize states are relevant:
 - `'ActivateChanges'` – applies the pending changes at runtime with no restart.
 - `'WarmStart'` – applies the pending changes and performs a soft reboot of the device.
 - `'ColdStart'` – applies the pending changes and performs a hard reboot (full power-cycle restart) of the device. Behaviourally equivalent to `WarmStart` from the perspective of applying network port changes; both must be handled if the device supports the `ReinitializeDevice` service (required by BTL).

This behaviour applies to both the BACnet IP network port and to each virtual network port object that represents a virtual network.

Step 1 – Extend the Database Model

1.1 Add `prevNetworkNumber` to the base network port class

The database needs to track the *previous* network number so that `ActivateChanges` can detect what changed and issue the correct BACnet broadcasts. Add `prevNetworkNumber` (and `networkNumberQuality` if not present) to `ExampleDatabaseNetworkPortBase` in `'CASBACnetStackExampleDatabase.h'`:

```

// CASBACnetStackExampleDatabase.h lines 55-62
class ExampleDatabaseNetworkPortBase : public
ExampleDatabaseBaseObject
{
public:
    bool        changesPending;
    uint16_t    networkNumber;
    uint8_t     networkNumberQuality; // unknown(0), learned(1),
learned-configured(2), configured(3)
    uint16_t    prevNetworkNumber;
};

```

`ExampleDatabaseNetworkPortIpv4` inherits from this base class and therefore automatically gains the same fields.

1.2 Initialise the fields in `ExampleDatabase::Setup()`

In `CASBACnetStackExampleDatabase.cpp`, make sure every field is initialised before use:

```

// CASBACnetStackExampleDatabase.cpp lines 49-54
// BACnet IP network port
this->networkPort.instance           = 1;
this->networkPort.objectName         = "Network Port for Ipv4";
this->networkPort.changesPending     = false;
this->networkPort.networkNumber      = 0;
this->networkPort.networkNumberQuality = 0; // unknown - not yet
configured
this->networkPort.prevNetworkNumber  = 0;

```

1.3 Initialise virtual network port fields in `LoadVirtualDevices()`

```

// CASBACnetStackExampleDatabase.cpp lines 62-69
ExampleDatabaseNetworkPortBase virtualNetworkPort;
virtualNetworkPort.changesPending     = false;
virtualNetworkPort.networkNumberQuality = 3; // configured
virtualNetworkPort.instance           = STARTING_VIRTUAL_NETWORK
+ (networkIndex * VIRTUAL_NETWORK_OFFSET);
uint16_t networkNumber                = STARTING_VIRTUAL_NETWORK
+ (networkIndex * VIRTUAL_NETWORK_OFFSET);
virtualNetworkPort.networkNumber      = networkNumber;
virtualNetworkPort.prevNetworkNumber  = networkNumber; //

```

```
matches current value - no pending change
    this->virtualNetworkPorts[virtualNetworkPort.instance] =
virtualNetworkPort;
```

Step 2 – Enable WriteProperty on the Main Device

The `WriteProperty` service must be enabled on the main (router) device so that BACnet clients can write properties:

```
// BACnetVirtualDevicesServerExampleCPP.cpp lines 241-246
std::cout << "Enabling WriteProperty... ";
if (!fpSetServiceEnabled(g_database.mainDevice.instance,
CASBACnetStackExampleConstants::SERVICE_WRITE_PROPERTY, true)) {
    std::cerr << "Failed to enable the WriteProperty" << std::endl;
    return false;
}
std::cout << "OK" << std::endl;
```

You also need `ReinitializeDevice` so that `ActivateChanges` / `WarmStart` can be sent:

```
// BACnetVirtualDevicesServerExampleCPP.cpp lines 248-253
std::cout << "Enabling ReinitializeDevice... ";
if (!fpSetServiceEnabled(g_database.mainDevice.instance,
CASBACnetStackExampleConstants::SERVICE_REINITIALIZE_DEVICE, true)) {
    std::cerr << "Failed to enable the ReinitializeDevice" <<
std::endl;
    return false;
}
std::cout << "OK" << std::endl;
```

Step 3 – Register the Set-Property Callback

Register the unsigned-integer write callback so the stack can deliver write requests for `Network_Number`:

```
// BACnetVirtualDevicesServerExampleCPP.cpp line 206  
fpRegisterCallbackSetPropertyUnsignedInteger(CallbackSetPropertyUInt);
```

Step 4 – Handle the Write in `CallbackSetPropertyUInt`

When the stack receives a `WriteProperty` for `Network_Number` it calls this callback. There are two cases to handle:

- **BACnet IP network port** – identified by `objectInstance == g_database.networkPort.instance`.
- **Virtual network port** – identified by a matching key in `g_database.virtualNetworkPorts`.

```
// BACnetVirtualDevicesServerExampleCPP.cpp lines 744-773  
bool CallbackSetPropertyUInt(  
    const uint32_t deviceInstance,  
    const uint16_t objectType,  
    const uint32_t objectInstance,  
    const uint32_t propertyIdentifier,  
    const uint32_t value,  
    const bool     useArrayIndex,  
    const uint32_t propertyArrayIndex,  
    const uint8_t  priority,  
    uint32_t*      errorCode)  
{  
    if (propertyIdentifier ==  
CASBACnetStackExampleConstants::PROPERTY_IDENTIFIER_NETWORK_NUMBER) {  
        if (deviceInstance == g_database.mainDevice.instance) {  
  
            // Reject values outside the valid BACnet network  
number range  
            if (value > 65535) {  
                *errorCode =  
CASBACnetStackExampleConstants::ERROR_VALUE_OUT_OF_RANGE;  
                return false;  
            }  
        }  
    }  
}
```

```

        // --- BACnet IP network port ---
        if (objectInstance == g_database.networkPort.instance)
        {
            if (objectType ==
CASBACnetStackExampleConstants::OBJECT_TYPE_NETWORK_PORT) {
                std::cout << "Received request to set Network
Number property of the Network Port Object."
                << " value=[" << value << "],
priority=[" << (int)priority << "]" << std::endl;

                g_database.networkPort.prevNetworkNumber    =
g_database.networkPort.networkNumber;
                g_database.networkPort.networkNumber        =
value;
                g_database.networkPort.networkNumberQuality =
3; // configured
                g_database.networkPort.changesPending       =
true;
                return true;
            }
        }

        // --- Virtual network port ---
        else if
(g_database.virtualNetworkPorts.count(objectInstance) > 0) {
            g_database.virtualNetworkPorts[objectInstance].prevNetworkNumber    =
g_database.virtualNetworkPorts[objectInstance].networkNumber;
            g_database.virtualNetworkPorts[objectInstance].networkNumber        =
value;
            g_database.virtualNetworkPorts[objectInstance].networkNumberQuality =
3;
            g_database.virtualNetworkPorts[objectInstance].changesPending       =
true;
            return true;
        }
    }
    return false;
}

```

Key points

- Validate `value <= 65535` first and return `ERROR_VALUE_OUT_OF_RANGE` if not — `Network_Number` is a `Unsigned16` in the BACnet standard.
- Save the current value to `prevNetworkNumber` *before* overwriting `networkNumber`.
- Set `networkNumberQuality` to 3 (configured) to reflect that the value was explicitly set.

- Set `changesPending = true`. The stack reads this flag via `CallbackGetPropertyBool` and exposes it on the `Changes_Pending` property.
 - Do **not** apply the change yet — that happens in `ActivateChanges`.
-

Step 5 – Serve `Changes_Pending` via `CallbackGetPropertyBool`

The stack will call this callback when a client reads the `Changes_Pending` property. Handle both the BACnet IP port and virtual ports:

```
// BACnetVirtualDevicesServerExampleCPP.cpp lines 584-600
bool CallbackGetPropertyBool(
    uint32_t deviceInstance,
    uint16_t objectType,
    uint32_t objectInstance,
    uint32_t propertyIdentifier,
    bool* value,
    bool useArrayIndex,
    uint32_t propertyArrayIndex)
{
    if (propertyIdentifier ==
CASBACnetStackExampleConstants::PROPERTY_IDENTIFIER_CHANGES_PENDING) {

        // BACnet IP network port
        if (objectType ==
CASBACnetStackExampleConstants::OBJECT_TYPE_NETWORK_PORT &&
            objectInstance == g_database.networkPort.instance) {
            *value = g_database.networkPort.changesPending;
            return true;
        }

        // Virtual network ports
        if (objectType ==
CASBACnetStackExampleConstants::OBJECT_TYPE_NETWORK_PORT) {
            if
(g_database.virtualNetworkPorts.count(objectInstance) > 0) {
                *value =
g_database.virtualNetworkPorts[objectInstance].changesPending;
                return true;
            }
        }
    }
    return false;
}
```

Step 6 – Apply Changes in `ActivateChanges()`

`ActivateChanges` is called after `ReinitializeDevice` (`ActivateChanges`) is acknowledged. It must:

1. Detect which network numbers changed by comparing `prevNetworkNumber` with `networkNumber`.
2. Send a `NetworkNumberIs` broadcast for the BACnet IP port if its number changed.
3. Call `fpUpdateVirtualNetworkNumber` for each virtual port that changed, then send a single `IAmRouterToNetwork` broadcast.

```
// BACnetVirtualDevicesServerExampleCPP.cpp lines 391-431
void ActivateChanges() {
    std::cout << "FYI: Activating Changes for NetworkPort
Objects..." << std::endl;

    // --- BACnet IP network port ---
    if (g_database.networkPort.prevNetworkNumber !=
g_database.networkPort.networkNumber) {
        std::cout << "Network Port network number changed. Sending
NetworkNumberIs broadcast with new network number=["
                << g_database.networkPort.networkNumber << "]" <<
std::endl;
        uint8_t connectionString[6];
        memcpy(connectionString,
g_database.networkPort.BroadcastIPAddress, 4);
        connectionString[4] =
g_database.networkPort.BACnetIPUDPPort / 256;
        connectionString[5] =
g_database.networkPort.BACnetIPUDPPort % 256;
        if (!fpSendNetworkNumberIs(
            g_database.networkPort.networkNumber,
            g_database.networkPort.networkNumberQuality,
            connectionString, 6,
            CASBACnetStackExampleConstants::NETWORK_TYPE_IP,
            true, 65535, NULL, 0)) {
            std::cerr << "Unable to send NetworkNumberIs broadcast
for network number=["
                    << g_database.networkPort.networkNumber <<
                    "]" << std::endl;
        }
        g_database.networkPort.prevNetworkNumber =
g_database.networkPort.networkNumber;
    }

    // --- Virtual network ports ---
    bool virtualNetworkPortChanged = false;
    std::map<uint32_t, ExampleDatabaseNetworkPortBase>::iterator
```

```

networkPortIt;
    for (networkPortIt = g_database.virtualNetworkPorts.begin();
         networkPortIt != g_database.virtualNetworkPorts.end();
         ++networkPortIt) {

        if (networkPortIt->second.prevNetworkNumber != 0 &&
            networkPortIt->second.prevNetworkNumber !=
networkPortIt->second.networkNumber) {

            std::cout << "Virtual Network Port [" << networkPortIt-
>first
                        << "] network number changed to ["
                        << networkPortIt->second.networkNumber << "]"
<< std::endl;

            virtualNetworkPortChanged = true;

            if (!fpUpdateVirtualNetworkNumber(networkPortIt-
>second.prevNetworkNumber,
                                              networkPortIt-
>second.networkNumber)) {
                std::cerr << "Unable to update the virtual network
number for virtual network port ["
                        << networkPortIt->first << "]" <<
std::endl;
            }
            networkPortIt->second.prevNetworkNumber =
networkPortIt->second.networkNumber;
        }
    }

    if (virtualNetworkPortChanged) {
        std::cout << "Sending IAmRouterToNetwork broadcast with new
network numbers..." << std::endl;
        uint8_t connectionString[6];
        memcpy(connectionString,
g_database.networkPort.BroadcastIPAddress, 4);
        connectionString[4] =
g_database.networkPort.BACnetIPUDPPort / 256;
        connectionString[5] =
g_database.networkPort.BACnetIPUDPPort % 256;
        if (!fpSendIAmRouterToNetwork(connectionString, 6,
CASBACnetStackExampleConstants::NETWORK_TYPE_IP,
                                true, 65535, NULL, 0)) {
            std::cerr << "Unable to send IAmRouterToNetwork
broadcast" << std::endl;
        }
    }
}

```

Why `fpUpdateVirtualNetworkNumber`?

The CAS BACnet Stack internally routes messages using the virtual network number. When the number changes, the stack's routing table must be updated via this function before the new number is active.

Step 7 – Handle ReinitializeDevice to Clear Changes_Pending

The `ReinitializeDevice` service supports three states that are relevant to network port changes:

State	Meaning	Action
<code>ActivateChanges</code>	Apply pending changes at runtime — no restart	Clear <code>changesPending</code> , signal deferred <code>ActivateChanges()</code> call
<code>WarmStart</code>	Apply pending changes then soft-reboot the device	Clear <code>changesPending</code> , signal deferred <code>WarmStart()</code> call
<code>ColdStart</code>	Apply pending changes then hard-reboot (power-cycle) the device	Same handling as <code>WarmStart</code>

`WarmStart` and `ColdStart` are behaviourally equivalent from the perspective of applying network port changes — both apply pending changes and then restart the device. The difference is the type of restart: `WarmStart` is a software reboot and `ColdStart` is a full hardware power-cycle. **This example does not handle 'ColdStart'**, but a production implementation must, as BTL requires a device to handle both `WarmStart` and `ColdStart` if it supports the `ReinitializeDevice` service.

In `CallbackReinitializeDevice`, clear `changesPending` for all network ports and set the appropriate flag so that `ActivateChanges` (or `WarmStart`) is processed outside the callback after the stack sends its `SimpleAck`:

```
// BACnetVirtualDevicesServerExample.cpp lines 775-836
bool CallbackReinitializeDevice(
    const uint32_t deviceInstance,
    const uint32_t reinitializedState,
    const char* password,
    const uint32_t passwordLength,
    uint32_t* errorCode)
{
    // Validate password
    if (password == NULL || passwordLength == 0) {
        *errorCode =
CASBACnetStackExampleConstants::ERROR_PASSWORD_FAILURE;
```

```

        return false;
    }
    if (strncmp(password, "12345", passwordLength) != 0) {
        *errorCode =
CASBACnetStackExampleConstants::ERROR_PASSWORD_FAILURE;
        return false;
    }

    if (reinitializedState ==
CASBACnetStackExampleConstants::REINITIALIZED_STATE_ACTIVATE_CHANGES) {

        // Clear Changes_Pending on all network ports
        g_database.networkPort.changesPending = false;
        for (auto& kv : g_database.virtualNetworkPorts) {
            kv.second.changesPending = false;
        }

        // Signal the main loop to call ActivateChanges() after the
SimpleAck is sent
        g_activateChanges = true;
        return true;
    }
    else if (reinitializedState ==
CASBACnetStackExampleConstants::REINITIALIZED_STATE_WARM_START) {

        g_database.networkPort.changesPending = false;
        for (auto& kv : g_database.virtualNetworkPorts) {
            kv.second.changesPending = false;
        }

        g_warmStart = true;
        g_warmStartTimer = time(0);
        return true;
    }
    else {
        *errorCode =
CASBACnetStackExampleConstants::ERROR_OPTIONAL_FUNCTIONALITY_NOT_SUPPORTED;
        return false;
    }
}

```

Important: Never apply changes or restart the device directly inside the callback. Return true first so the stack can send the SimpleAck, then act on the flag in the main loop.

ColdStart not implemented: The else branch above returns ERROR_OPTIONAL_FUNCTIONALITY_NOT_SUPPORTED for all other states, including ColdStart. A BTL-compliant production device must add a ColdStart branch that mirrors the WarmStart handling (clear changesPending, set a deferred flag, then perform a full power-cycle restart after the SimpleAck is sent).

Step 8 – Handle Deferred Activation in the Main Loop

The `g_activateChanges` and `g_warmStart` flags are consumed in the main loop:

```
// BACnetVirtualDevicesServerExampleCPP.cpp lines 153-162
for (;;) {
    // Warm start (deferred 3 seconds to allow SimpleAck to be
sent)
    if (g_warmStart && g_warmStartTimer + 3 < time(0)) {
        WarmStart();
    }

    // Activate changes
    if (g_activateChanges) {
        g_activateChanges = false;
        ActivateChanges();
    }

    fpTick();
    // ...
}
```

Understanding `ActivateChanges()` and `WarmStart()`

This section walks through exactly what each function does in this example and highlights what a production implementation would need to add or change.

`ActivateChanges()` – Apply pending changes at runtime (no restart)

`ActivateChanges()` (lines 391–431 in `BACnetVirtualDevicesServerExampleCPP.cpp`) is called from the main loop immediately after the stack has sent the `SimpleAck` for a `ReinitializeDevice` (`ActivateChanges`) request. It applies any pending `Network_Number` changes without restarting the device.

What this example does, step by step:

1. **BACnet IP network port** – Compares `prevNetworkNumber` with `networkNumber`. If they differ, builds the IP broadcast connection string inline, calls `fpSendNetworkNumberIs()` so every device on the BACnet/IP network learns the port's new network number, then resets `prevNetworkNumber = networkNumber` so a second call does not re-broadcast.
2. **Virtual network ports** – Iterates every entry in `g_database.virtualNetworkPorts`. For each port where `prevNetworkNumber` is non-zero and differs from `networkNumber`, calls `fpUpdateVirtualNetworkNumber(prevNetworkNumber, networkNumber)` to update the CAS BACnet Stack's internal routing table, sets a `virtualNetworkPortChanged` flag, then resets `prevNetworkNumber = networkNumber`.
3. **'IAmRouterToNetwork' broadcast** – If at least one virtual port changed, builds the broadcast connection string inline and sends a single `fpSendIAmRouterToNetwork()` so neighbouring routers and devices learn the updated network-to-router mapping.

What a production implementation must also do:

- **Persist the new values** – Write the updated network numbers to non-volatile storage (EEPROM, flash, config file, etc.) so they survive a power cycle. In this example, values are only held in RAM and are lost on restart.

WarmStart() – Apply pending changes then soft-reboot

`WarmStart()` (lines 373–389 in `BACnetVirtualDevicesServerExampleCPP.cpp`) is called from the main loop after a 3-second deferred timer expires, giving the stack time to transmit the `SimpleAck` for the `ReinitializeDevice (WarmStart)` request before the restart begins.

What this example does, step by step:

1. **'fpReset()'** – Tears down and fully reinitialises the CAS BACnet Stack, clearing all registered objects, callbacks, and internal state.
2. **'g_database.ReloadVirtualDevices()'** – Clears the `virtualDevices` map and rebuilds it by iterating `virtualNetworkPorts`, using each port's current `networkNumber`

value. This means any `networkNumber` that was written before the warm start is automatically picked up when the objects are re-registered in the next step.

3. ``RegisterCallbacks()`` – Re-registers all message, system-time, get-property, set-property, and remote-device-management callbacks with the freshly reset stack instance.

4. ``SetupDevice()`` – Re-adds the main device, network port object, virtual networks, virtual devices, and analog input objects to the stack. Returns `false` on failure, in which case `WarmStart()` logs an error and aborts.

5. ``SendIAM()`` – Broadcasts `IAm`, `IAmRouterToNetwork`, and `NetworkNumberIs` to re-announce the device and its updated routing information to the rest of the network. Returns `false` on failure, in which case `WarmStart()` logs an error and aborts.

What a production implementation must also do:

- **Persist before resetting** – Write all pending `networkNumber` values to non-volatile storage *before* calling `fpReset()`. In this example, `ReloadVirtualDevices()` reads from the in-RAM database (which still holds the written values), but on a real device the startup path typically reads from persistent storage, so values must be saved there first.
- **‘ColdStart’ parity** – A `ColdStart` handler must perform the same persistence step, but instead of calling `fpReset()` in-process it must trigger a full hardware power-cycle (e.g., a watchdog reset or a BSP restart call). The remaining steps — reload, re-register, re-setup, re-announce — then happen naturally as part of normal startup after the reboot.

Step 9 – Send `NetworkNumberIs` on Startup

When the device starts up with a configured network number (quality = 3), broadcast a `NetworkNumberIs` so the rest of the network learns the mapping immediately:

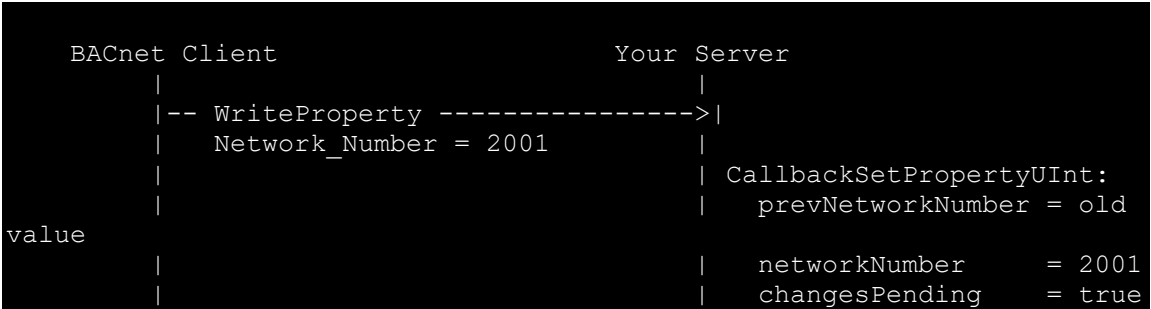
```
// BACnetVirtualDevicesServerExample.cpp lines 362-368
// In SendIAM() - after sending IAmRouterToNetwork
if (g_database.networkPort.networkNumberQuality != 0) {
    if (!fpSendNetworkNumberIs(
        g_database.networkPort.networkNumber,
        g_database.networkPort.networkNumberQuality,
        connectionString, 6,
        CASBACnetStackExampleConstants::NETWORK_TYPE_IP,
        true, 65535, NULL, 0)) {
        std::cerr << "Unable to send NetworkNumberIs broadcast for
network number=["
                    << g_database.networkPort.networkNumber << "]" <<
        std::endl;
        return false;
    }
}
```

```
    }  
}
```

Summary of Changes

File	What changes
CASBACnetStackExampleDatabase.h	ExampleDatabaseNetworkPortBase gains prevNetworkNumber and networkNumberQuality fields
CASBACnetStackExampleDatabase.cpp	Setup() and LoadVirtualDevices() initialise the new fields, including prevNetworkNumber = 0
BACnetVirtualDevicesServerExampleCPP.cpp	SetupDevice() enables WriteProperty and ReinitializeDevice; RegisterCallbacks() registers CallbackSetPropertyUInt; CallbackSetPropertyUInt handles PROPERTY_IDENTIFIER_NETWORK_NUMBER for both port types; CallbackGetPropertyBool serves Changes_Pending; CallbackReinitializeDevice clears flags and sets deferred-action globals; ActivateChanges() calls fpUpdateVirtualNetworkNumber and sends the appropriate broadcasts; SendIAM() sends NetworkNumberIs on startup

Property Flow Diagram



```

|<-- SimpleAck -----|
|
|
|-- ReadProperty Changes_Pending ->| CallbackGetPropertyBool
→ true
|<-- TRUE -----|
|
|-- ReinitializeDevice ----->|
| (ActivateChanges, pw=12345) |
|
CallbackReinitializeDevice:
|
| changesPending = false
| g_activateChanges = true
|<-- SimpleAck -----|
|
| Main loop detects
g_activateChanges:
|
fpUpdateVirtualNetworkNumber(old, 2001)
|
fpSendIamRouterToNetwork(...)
|<-- IamRouterToNetwork broadcast-|

```